# Four Detectors That Turned Out to Be One: What We Learned Trying to Catch Fake Speech

A. Harisha
Sahyadri College of Engineering and Management
harisha@sahyadri.edu.in

October 27, 2026

**Abstract**

Software can now generate speech that sounds like a real person, so we need software that can tell real speech from fake. The usual way to build a strong detector is to build several different ones and average their answers, hoping their mistakes differ. We built four detectors that were different by design, and found they made almost the same mistakes: together they were worth about one detector, not four. We also found that the standard way of choosing which detector to use does not work on this data, and that the detector which scores best on the official test is the one that performs worst on real audio collected from the internet. Our best honestly-chosen system gets 3.21 out of 100 wrong at its break-even point, which is good but behind the best published system at 2.59.

## 1 What this is about

A computer can now be given a few seconds of someone's voice and produce new sentences in that voice. This is useful for accessibility and dubbing, and harmful for fraud and impersonation. So researchers build *detectors*: programs that listen to a recording and output a number saying how likely it is to be genuine.

**How detectors are scored.** We mostly use one number, the **equal error rate**, or EER. Imagine turning a dial on the detector. Turn it one way and it becomes suspicious, catching every fake but also wrongly flagging real speech. Turn it the other way and it trusts everything, never flagging real speech but missing fakes. Somewhere in the middle, the two kinds of mistake happen equally often. That percentage is the EER. **Lower is better:** an EER of 3% means that at the balanced setting, three out of a hundred judgements are wrong. We also report a cost-weighted score called minDCF, which penalises the more expensive mistake more heavily.

**The data.** We use ASVspoof 5 [Wang et al., 2024a], the standard public benchmark. It has one property that makes it honest: the fake-speech systems used in the training data are *different* from the ones in the test data. A detector cannot pass by memorising the particular fakes it was shown. Table 1 describes it.

## 2 What we built

Every one of our detectors has the same two stages (Figure 1).

The **first stage** is a large pre-trained model called WavLM [Chen et al., 2022]. It was trained on 94,000 hours of ordinary speech, with no knowledge of fakes at all, and it turns a recording into a rich numerical description. We barely change it; we attach a small number

**Table 1:** The data we used. The fake-generation systems in each part do not overlap, so the test always uses fakes the detector has never seen.

| Part | Recordings | Real speech | What it is for |
|---|---|---|---|
| Training | 182,357 | 10% | teaching the detector |
| Development | 140,950 | 22% | choosing between detectors |
| Test | 680,774 | 20% | the final score, used once |
| In-the-Wild | 31,779 | 63% | audio from the internet, a reality check |

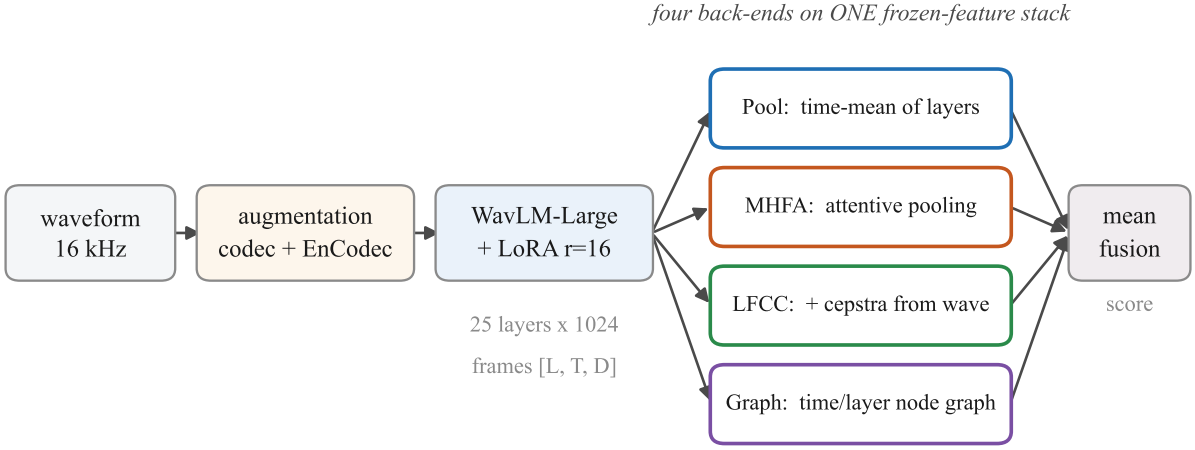



**Figure 1:** All four detectors share the first stage and differ only in the last box before the score.

of extra parameters using a technique called LoRA [Hu et al., 2022], which lets us adjust a 300-million-parameter model by training only about 3 million numbers.

The **second stage**, which we call the back-end, is a small network that reads that description and produces the final score. This is the part we varied. We built four:

- **Pool** — the ordinary approach. It averages the description over the whole recording, then decides. Averaging over time throws away *when* things happened.
- **MHFA** — instead of averaging, it learns *where in the recording to look*.
- **LFCC** — it adds a completely separate description of the sound, a classical one based on frequency content, computed directly from the waveform rather than from WavLM.
- **Graph** — it treats pieces of the recording as points in a network and lets them exchange information, an idea borrowed from a detector called AASIST [Jung et al., 2022].

Everything else is held identical: same training data, same random seed, same schedule. So any difference between them comes from the back-end alone.

## 3 Finding 1: the four detectors were really one detector

The reason to build several detectors is that averaging helps *only if they make different mistakes*. If two detectors are wrong about exactly the same recordings, averaging them changes nothing.

So we measured how different they actually were. Think of each detector's set of scores as a direction. If the four directions point different ways, we have four detectors. If they all point the same way, we have one.

The result is Figure 2. A single shared direction accounts for **95.5%** of everything the four detectors do, and all four contribute to it almost exactly equally. By a standard measure of how many independent things are in a set, our four detectors are worth **1.10**.

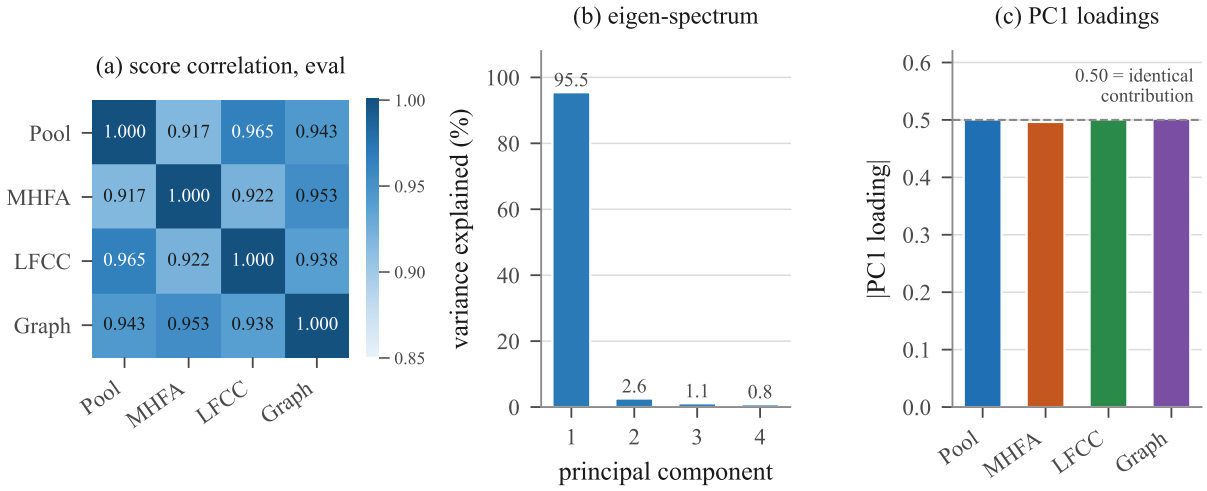



**Figure 2:** (a) How similar the detectors' scores are — 0.92 to 0.97, where 1.00 would mean identical. (b) One shared direction explains 95.5% of their behaviour. (c) All four contribute to it equally.

We changed the way the model summarises time, the kind of sound description it reads, the shape of its network, and even the large pre-trained model underneath. All of that moved the count from 1.00 to 1.10.

The one change that helped at all was MHFA — the detector that looks at *when* things happen instead of averaging them away. Averaging a four-second clip into a single summary turns out to throw away more than we expected. That change bought a small but real improvement, and it came from the 3% of behaviour the detectors did *not* share.

## 4 Finding 2: we cannot tell which detector is best

Standard practice is: train on part of the data, use a second part (the *development* set) to choose your best system, then report its score on the test set. Using the test set to choose would be cheating, because then the score no longer predicts anything new.

We followed that rule. It did not work.

We had 15 possible combinations of our four detectors. We ranked them by their development score and compared that to their test score. **The two rankings are unrelated** — Figure 3 shows the correlation is +0.01, which is statistically indistinguishable from pure chance.

**Why.** The training and development recordings are all clean, uncompressed audio. Three quarters of the test recordings have been through a compression codec — the kind of processing a phone call or a voice message applies. So the development set simply never shows us the thing the test set is actually judging.

We tried the obvious fix: we added compressed audio to the development set. The ranking improved, but not enough; the correlation rose to +0.24, still within chance.

**What it costs us, stated openly.** Our rule picks a system scoring **3.21**. The best system we actually have scores **2.93**, and our rule ranks it seventh out of fifteen. We report **both** numbers, in Table 2, and we label the second one clearly as a system we could not have chosen honestly. The 0.28 gap between them is not a flaw in our detectors. It is a measurement of how poor the selection procedure is.

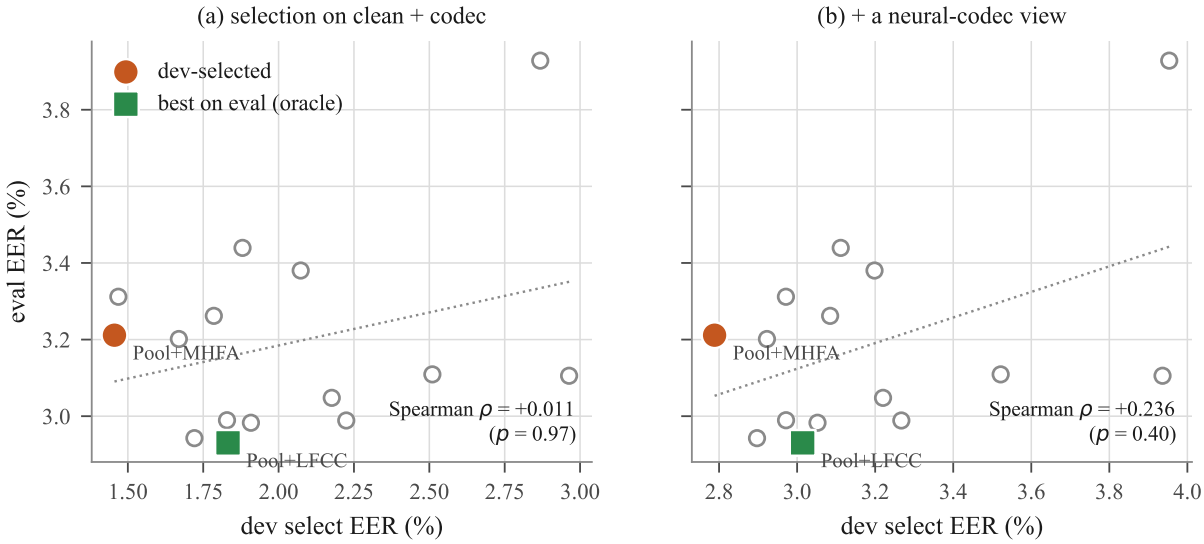



**Figure 3:** Each dot is one combination of detectors. If the development set were useful, the dots would fall along a rising line. They do not.

**Table 2:** Results on the test set. Lower is better. The oracle row is what we would have picked if we were allowed to cheat by looking at the test set; we show it for honesty, not as our result.

| System | How it was chosen | EER (%) | minDCF |
|---|---|---|---|
| Best published system, 12 detectors [Wang et al., 2024b] | — | 2.59 | 0.0750 |
| Pool + LFCC | *oracle — not claimed* | *2.93* | *0.0819* |
| **Pool + MHFA** | **our rule** | **3.21** | **0.0905** |
| Pool alone | — | 3.31 | 0.0942 |
| Another published system [Xu et al., 2024] | — | 4.04 | 0.115 |

## 5 Finding 3: doing well on the test says little about the real world

The test set is large and carefully built, but every recording in it was made by one project under controlled conditions. So we also ran our detectors on In-the-Wild: about 32,000 clips of real and fake speech of public figures, collected from the internet, with no labels about how they were recorded.

Two things happened (Figure 4).

**Everything got much worse.** Error rates roughly tripled, from 3–4% to 8–12%. That is normal in this field but rarely stated plainly.

**The order flipped.** LFCC, our *best* detector on the official test, was our *worst* on internet audio. MHFA, which was third on the official test, was clearly the best in the wild — 8.51 against 11.25 for the next one.

A likely explanation is that LFCC partly learned the *recording conditions* of this particular dataset rather than what makes speech fake. Recording conditions do not carry over to audio from somewhere else. We cannot prove this; we can only show that the flip happens.

This has an awkward implication. Chasing the last fraction of a percent on the official test would mean preferring exactly the detector that does worst on real material.

## 6 Finding 4: all of these detectors are easy to fool deliberately

Everything above concerns ordinary fake speech. A different question is what happens when someone *attacks* the detector on purpose, adding a small amount of carefully computed noise

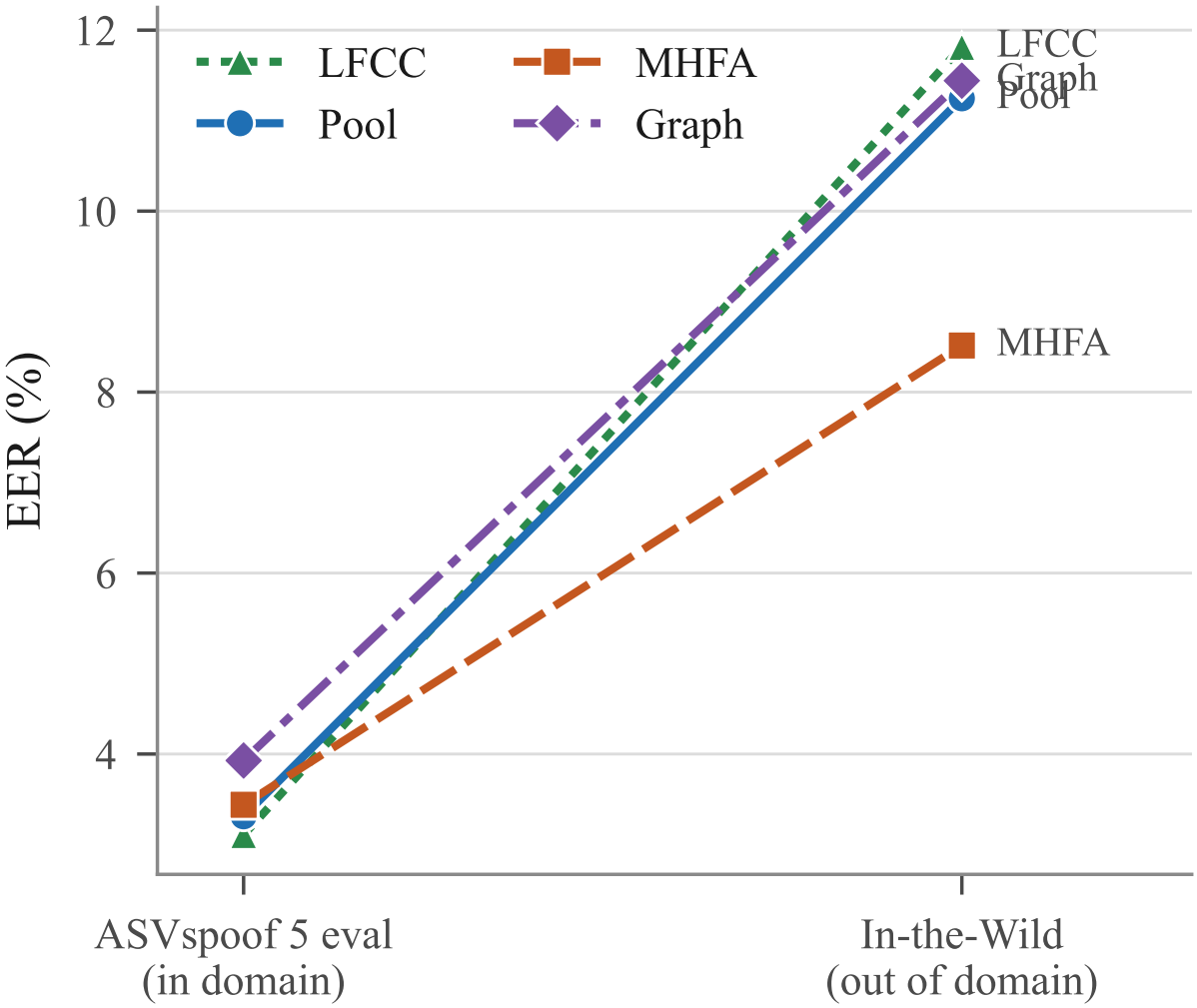



**Figure 4:** Each line is one detector, from the official test on the left to internet audio on the right. The lines cross: the winner on the left is the loser on the right.

designed to flip its answer.

We tried this with noise quiet enough to be about one thousandth of the signal's power. If the attacker can see inside the detector, **it succeeds on essentially every recording**, for every version we built: a single detector, a group of four, and across two different pre-trained models. An attacker who can only poke the detector from outside, with a limited number of tries, succeeds about 1% of the time.

We tried two defences, both of which train the detector on attacked examples. The first defended the wrong part of the system; it failed completely and made ordinary accuracy much worse. The second defended the right part, and it is the only thing in this project that made an attacker's job harder: the success rate fell from 100% to about 95% when the attacker can see inside the detector, and from 100% to 74% when the attacker must work through a copy of it.

That sounds like progress until you read the bill. **Ordinary accuracy became twice as bad**, from 3.31% to 7.09%, in exchange for taking five percentage points off an attack that still works nineteen times in twenty. Nobody would ship that exchange. Our honest conclusion is that these detectors cannot currently be protected from a determined attacker at an accuracy anyone would accept. We think that is worth saying plainly, because the alternative is a reader assuming the problem has been solved.

## 7 What we would tell someone starting this

1. **Building more models is not the same as building different models.** Check how much your models actually disagree before assuming a group of them will help. It is a short calculation and it would have saved us weeks.
2. **Check that your development set contains the conditions your test set judges.** If it does not, your procedure for choosing a system is making a decision at random, and you will not notice.
3. **Test on something you did not build.** The gap between a curated benchmark and found audio was a factor of three for us, and it reordered our conclusions.
4. **Report the system your rules chose, not the best one you found.** If they differ, that difference is itself worth reporting.

## 8 Honest limitations

We ran each system once, with a single random starting point. In earlier experiments on this code, simply changing that starting point moved results by 0.33 — larger than the 0.10 improvement we report from combining two detectors. So our improvement is real but not firmly established.

We trained on one dataset. The best published system trains on four. We therefore cannot separate "our design is worse" from "we showed our system less material".

The official test set contains 680,774 recordings, but they come from only 171,602 original utterances, each appearing once clean and about three times after compression. Statistical confidence intervals that treat the recordings as independent — including ours in earlier drafts — are roughly twice too narrow.

Finally, our Graph detector borrows one component from AASIST but is not AASIST, and nothing here should be read as a result about that system.

## 9 Summary

We set out to build a strong ensemble of fake-speech detectors and to beat the best published system. We did not beat it: our honestly-chosen system scores 3.21 against its 2.59.

What we found instead is, we think, more useful. Four detectors that differ in every architectural way we could think of behave as one. The standard procedure for choosing between them does not work on the standard dataset, and we can measure exactly how much that costs. And the detector that wins the benchmark is the one that fails hardest on real audio.

## References


Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, Jian Wu, Long Zhou, Shuo Ren, Yanmin Qian, Yao Qian, Jian Wu, Michael Zeng, Xiangzhan Yu, and Furu Wei. WavLM: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022. URL `https://arxiv.org/abs/2110.13900`. arXiv:2110.13900.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL `https://arxiv.org/abs/2106.09685`. arXiv:2106.09685.

Jee-weon Jung, Hee-Soo Heo, Hemlata Tak, Hye-jin Shim, Joon Son Chung, Bong-Jin Lee, Ha-Jin Yu, and Nicholas Evans. AASIST: Audio anti-spoofing using integrated spectro-temporal graph attention networks. In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 2405–2409, 2022. doi: 10.1109/ICASSP43922.2022.9747766. URL `https://arxiv.org/abs/2110.01200`. arXiv:2110.01200.

Xin Wang, Héctor Delgado, Hemlata Tak, Jee-weon Jung, Hye-jin Shim, Massimiliano Todisco, Ivan Kukanov, Xuechen Liu, Md Sahidullah, Tomi Kinnunen, Nicholas Evans, Kong Aik Lee, and Junichi Yamagishi. ASVspoof 5: Crowdsourced speech data, deepfakes, and adversarial attacks at scale. *arXiv preprint arXiv:2408.08739*, 2024a. URL `https://arxiv.org/abs/2408.08739`.

Yihao Wang, Yuxuan Fu, Zhiwei Cai, Shang Wang, Jiayi Hu, Jie Wang, Yuhao Lu, et al. USTC-KXDIGIT system description for ASVspoof5 challenge. *arXiv preprint arXiv:2409.01695*, 2024b. URL `https://arxiv.org/abs/2409.01695`. Open condition, track 1: 2.59% EER / 0.0750 minDCF; 12 fused systems.

Yuxiong Xu, Jiafeng Zhong, Sengui Zheng, Zefeng Liu, and Bin Li. SZU-AFS antispoofing system for the ASVspoof 5 challenge. In *Proc. ASVspoof Workshop*, 2024. URL `https://www.isca-archive.org/asvspoof_2024/xu24_asvspoof.pdf`. Open condition: 4.04% EER / 0.115 minDCF.